Why Finetuning Was Not Possible in RNN/LSTM Model?

Question:
Does a trained model for text classification directly usable for tasks like summarization?

Answer:
No, you cannot directly fine-tune an RNN/LSTM text classification model (e.g., IMDB sentiment) for a different task like summarization.

Why Not Directly Fine-Tune?

1. Task-Type Mismatch

Task	Input	Output	Objective
Classification	Text (sequence)	Class label (e.g., 0 or 1)	Predict class
Summarization	Text (sequence)	Text (sequence)	Generate sequence

- Classification uses LSTM → Dense → Softmax/Sigmoid.
- Summarization needs Encoder-Decoder (sequence-to-sequence).
-

So your IMDB LSTM model:

Input (sequence) → LSTM → Dense(1) → Sigmoid

has no decoder, no autoregression, and cannot generate sequences — hence, cannot summarize.

2. Architectural Mismatch

Summarization needs:

- An encoder (to read and understand input)
- A decoder (to generate summary word-by-word)
- Teacher forcing and autoregressive generation loop
- Typically attention mechanisms (Bahdanau or Luong attention with LSTM)

So classification models lack half the pipeline required for summarization.

Note: But you can reuse the LSTM encoder part, with major architectural changes — especially converting it into a Seq2Seq model.

3. Vocabulary Differences

- IMDB sentiment model may use a limited domain-specific vocab (e.g., emotional language).
- Let's assume summarization on a different dataset (Google News) needs a larger, richer vocabulary (news terms, entities, etc.).
- OOV word handling is also different.

4. Performance & Objective Function Mismatch

Task	Loss Function
Classification	Binary Cross-Entropy / Categorical Cross-Entropy
Summarization	Cross-Entropy over time steps (sequence-level loss)

Fine-tuning a classifier for summarization would yield garbage output, because:

- The model has never learned sequence generation.
- It has no attention, decoder, or positional memory.

5. What You Can Reuse (Optionally)

Component	Reusability
LSTM Encoder Weights	✅ Possible
Embedding Layer	✅ If vocab overlaps

Final Dense Layer	❌ Must be replaced
Architecture	❌ Needs redesign (add decoder)

Limitation of Traditional RNN/LSTM Models

RNN/LSTM-based models struggled to handle such task diversity effectively. Why?

- ❌ There were no universal language understanding models.
- ❌ LSTMs had to be trained from scratch for each task, which required large labeled datasets and huge compute.
- ❌ These models lacked transferability across tasks.

ULMFiT Research (by Howard & Ruder, 2018)

The ULMFiT (Universal Language Model Fine-tuning) paper demonstrated a breakthrough:

- They first trained an LSTM-based encoder-decoder model on a language modeling / translation task (to teach the model general language patterns).
- Then, they fine-tuned this pre-trained model on downstream tasks like text classification.
- ✅ Result: It significantly outperformed previous task-specific models.

Still Not a Fully Generic Solution

Even though ULMFiT showed the power of pretraining + fine-tuning, there were major limitations:

- LSTM-based models are computationally inefficient on large-scale data.
- They lack scalability due to sequential processing.
- Cannot handle very long sequences or massive multi-task pretraining.

That's why later models like BERT, GPT, and T5 — which are transformer-based — replaced LSTMs as universal NLP architectures.

Key Insights

- RNN/LSTM models were task-specific and not scalable.
- ULMFiT introduced transfer learning in NLP using LSTM, which was a major breakthrough.
- But due to LSTM's limitations (slow, hard to scale), it could not become a generic model across all NLP tasks.
- Modern universal models succeeded because:
    - They used transformers (parallelizable).
    - They were pretrained on massive corpora.
    - They were fine-tunable across many downstream tasks.